

W15D04 — Distributed Tracing

ML Engineer Journey · Phase 4: Database & Observability · 18 Juni 2026

1. Positioning: Pilar Keempat Observability

Di W15D01–D03 kamu sudah membangun tiga pilar: **Logging** (tahu APA yang terjadi), **Request ID Tracing** (korelasi log dalam satu service), dan **Metrics/Prometheus** (tahu SEBERAPA SERING dan LAMA rata-rata). Tiga pilar itu kuat — tapi masih punya blind spot.

Bayangkan request masuk ke service A, A panggil B, B panggil C, C query database. Total latency 4 detik. Log di masing-masing service ada. Metrics menunjukkan p95 naik. Tapi *di mana* 4 detik itu habis? Di A? Di B? Di C? Di network? Di DB? Tiga pilar sebelumnya tidak bisa menjawab itu. **Distributed Tracing** hadir untuk menutup gap itu.

Pilar	Menjawab	Tools	Scope
Logging	APA yang terjadi?	structlog / JSON	Single service
Request ID	Korelasi log satu service	ContextVar + UUID	Single service
Metrics	SEBERAPA SERING & LAMA?	Prometheus / Histogram	Cross-instance, aggregate
Tracing	MENGAPA request ini lambat?	OpenTelemetry + Jaeger	Cross-service, per-request

2. Konsep Inti: Trace, Span, dan Context Propagation

Analogi

Bayangkan kamu memesan makanan via aplikasi delivery. Kamu klik 'Pesan' — di balik layar terjadi banyak hal: konfirmasi ke payment gateway, approval bank, notif ke restoran, kurir dijadwalkan. Kalau makanan terlambat 45 menit, kamu ingin tahu: *telat di mana?*

Satu 'perjalanan' dari klik sampai makanan tiba adalah sebuah **Trace**. Setiap langkah individual di dalamnya adalah sebuah **Span**.

Definisi Teknis

- **Trace** — keseluruhan perjalanan satu request, dari masuk pertama sampai response kembali ke client. Diidentifikasi oleh `trace_id` yang unik.
- **Span** — satu unit kerja dalam trace. Setiap function call, DB query, HTTP call ke service lain masing-masing jadi satu span. Punya `span_id` sendiri, terikat ke `trace_id` yang sama.
- **Parent-Child Span** — span bisa punya parent. Kalau service A panggil service B, span di B adalah child dari span di A.
- **Context Propagation** — `trace_id` dan `span_id` dikirim antar service lewat HTTP header `traceparent` (W3C standard). Inilah yang membedakan dari `request_id` di W15D02 yang hanya hidup dalam satu service.

3. Request ID (W15D02) vs Trace ID (W15D04)

Keduanya bisa dan **sebaiknya** dipakai bersamaan — bukan saling menggantikan. Inject `trace_id` ke dalam log agar satu request bisa dilacak dari dua arah sekaligus.

Aspek	Request ID (W15D02)	Trace ID (W15D04)
Generator	Kamu generate sendiri (UUID4)	OpenTelemetry generate otomatis
Scope	Satu service saja	Lintas service (multi-hop)
Propagasi	Tidak otomatis keluar service	Otomatis via traceparent header
Visualisasi	Log file / grep	Jaeger UI — timeline waterfall
Format	UUID bebas	W3C TraceContext standard
Tujuan	Korelasi log dalam satu service	Lacak latency bottleneck antar service
Analogi	Foto kejadian perkara	Rekaman CCTV penuh lintas lokasi

4. OpenTelemetry — Standar Industri

Dulu setiap vendor monitoring (Datadog, Jaeger, Zipkin, New Relic) punya format trace sendiri-sendiri. Seperti setiap negara punya colokan listrik berbeda — repot kalau pindah. **OpenTelemetry (OTel)** adalah adaptornya: satu format standar, bisa dikirim ke backend manapun.

Kamu instrument kode satu kali → output bisa dikirim ke Jaeger, Zipkin, Grafana Tempo, Datadog, dll tanpa ubah kode.

Komponen	Fungsi
opentelemetry-sdk	Core SDK, buat Tracer, manage span lifecycle
opentelemetry-instrumentation-fastapi	Auto-instrument semua route FastAPI tanpa kode tambahan
opentelemetry-exporter-otlp-proto-grpc	Kirim span ke Jaeger via gRPC (port 4317)
opentelemetry-instrumentation-httpx	Auto-inject traceparent header ke setiap HTTP call keluar
Jaeger (via Docker)	Backend visualisasi trace — waterfall UI di port 16686

5. BatchSpanProcessor vs SimpleSpanProcessor

Ini salah satu keputusan konfigurasi paling berdampak ke performa API. Pilihan yang salah bisa membuat setiap request sedikit lebih lambat karena tracing overhead — mengalahkan tujuan observability itu sendiri.

Aspek	SimpleSpanProcessor	BatchSpanProcessor
Cara kerja	Kirim ke Jaeger langsung setiap span selesai	Kumpulkan span di memori, kirim batch secara async
Blocking?	YA — blocking per span	TIDAK — async background thread
Dampak ke API	Setiap request menunggu network call ke Jaeger	Request tidak terpengaruh oleh export latency
Use case	Debugging lokal, single-span test	Development, staging, PRODUCTION
Rekomendasi	Hindari di production	Selalu pakai ini

Analogi: SimpleSpanProcessor seperti kasir yang mengirim struk ke kantor pusat sebelum melayani pelanggan berikutnya. BatchSpanProcessor mengumpulkan semua struk, lalu kurir mengantarkannya sekaligus di akhir hari — tanpa memblokir kasir.

6. Implementasi: tracing.py

File ini adalah satu-satunya tempat setup OpenTelemetry. Prinsip separation of concerns dari W11D04 — tracing config tidak boleh tersebar di berbagai file.

```
# tracing.py
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.resources import Resource

def setup_tracing(service_name: str) -> trace.Tracer:
    resource = Resource.create({ "service.name": service_name, # label di Jaeger UI
                                "service.version": "1.0.0",
                                "deployment.environment": "development", })
    provider = TracerProvider(resource=resource) # insecure=True untuk local dev –
    # production wajib TLS
    exporter = OTLPSpanExporter(endpoint="localhost:4317", insecure=True)
    # BatchSpanProcessor: kumpulkan dulu, kirim batch async
    provider.add_span_processor(BatchSpanProcessor(exporter))
    # Set sebagai global – semua kode di app bisa pakai tanpa inject manual
    trace.set_tracer_provider(provider)
    return trace.get_tracer(service_name)
```

Penjelasan Baris Kritis

- **Resource.create()** — metadata tentang service ini. Jaeger pakai ini untuk label di UI. Tanpa ini semua service akan tampil dengan nama generik.
- **TracerProvider** — pabrik pembuat tracer. Semua span di app lewat provider ini. Analogikan sebagai manajer yang tahu ke mana semua laporan harus dikirim.
- **BatchSpanProcessor(exporter)** — lihat tabel perbandingan di atas. Selalu pakai ini.
- **trace.set_tracer_provider(provider)** — daftarkan provider sebagai global. Tanpa ini, `trace.get_tracer()` di file lain akan mengembalikan no-op tracer yang tidak merekam apapun.

7. Membuat Span Manual

FastAPIInstrumentor membuat span otomatis untuk setiap route. Tapi untuk sub-operasi yang ingin diukur lebih detail (preprocessing, inference, DB query), kamu buat span manual.

```
tracer = trace.get_tracer(__name__)
async def analyze(features: list[float]) -> dict:
    # Span 1: Validasi with
    with tracer.start_as_current_span("validate_features") as span:
        count =
```

```
len(features) span.set_attribute("features.count", count) # metadata ke Jaeger if count
== 0: span.set_attribute("error", True) # mark span sebagai error
span.add_event("Validasi gagal: kosong") # event = timestamp dalam span raise
ValueError("Features tidak boleh kosong") span.add_event("Validasi sukses") # Span 2:
Komputasi – count dideklarasikan ulang (eksplisit, bukan implicit dependency) with
tracer.start_as_current_span("compute_stats") as span: count = len(features) #
eksplisit, tidak bergantung scope span sebelumnya mean_val = sum(features) / count
max_val = max(features) span.set_attribute("stats.mean", mean_val)
span.set_attribute("stats.max", max_val) span.add_event(f"Komputasi selesai. Max:
{max_val}") return {"mean": mean_val, "max": max_val}
```

Catatan Penting dari Code Review

- **Deklarasi ulang count** — Python with-block tidak punya scope sendiri, tapi implicit dependency antar span adalah code smell. Deklarasi ulang membuat setiap span bisa berdiri sendiri.
- **span.add_event()** — timestamp dalam span. Berguna untuk menandai titik tertentu dalam operasi panjang tanpa harus membuat child span baru.
- **span.set_attribute('error', True)** — convention OTel untuk menandai span bermasalah. Jaeger akan menampilkan span ini dengan warna merah.
- **context manager safety** — span otomatis ditutup saat keluar from with-block, termasuk saat exception. Sama prinsipnya dengan `.time()` di W15D03.

8. Fire-and-Forget: Background Task dalam Trace

Dari screenshot Jaeger: POST /predictions/ selesai dalam 172ms, tapi total trace duration 5.5 detik. Ini adalah pola **fire-and-forget** yang diimplementasikan dengan benar.

- 172ms = waktu interaksi pengguna: validasi Pydantic → simpan ke DB dengan status pending → kembalikan 202 Accepted (struk antrian).
- 5.5s = waktu operasional background: FastAPI BackgroundTask bekerja di latar belakang, `validate_features` (59ms) + `compute_stats` (200ms) + simulasi ML (5s).
- Span BackgroundTask tetap terikat ke trace yang sama karena BackgroundTasks mewarisi context snapshot saat dijadwalkan — perilaku yang sudah kamu pahami dari W15D02.

Untuk menambah informasi ke span background task yang dibuat otomatis oleh instrumentor:

```
async def run_heavy_ml_computation(request_id: uuid.UUID): # Ambil span yang sedang
aktif (dibuat otomatis oleh FastAPIInstrumentor) current_span = trace.get_current_span()
current_span.set_attribute("task.request_id", str(request_id))
current_span.set_attribute("task.type", "ml_computation") ...
```

9. Integrasi dengan Stack W15D01–D03

Tracing bukan pengganti logging dan metrics — ketiganya saling melengkapi. Cara terbaik mengintegrasikan:

- **Log + Trace ID** — inject `trace_id` ke dalam setiap log entry. Ketika alert dari Prometheus muncul, cari `trace_id` di log, lalu buka di Jaeger.

- **Metrics** → **Tracing workflow** — Prometheus memberi sinyal 'ada masalah' (p95 latency naik). Tracing memberi jawaban 'di mana masalahnya' (span mana yang lambat).
- **exclude /metrics dari instrumentor** — sama seperti W15D03, endpoint /metrics harus di-exclude agar scrape Prometheus tidak mengotori trace data.
FastAPIInstrumentor(excluded_urls='/metrics,/health').

10. Knowledge Check — Jawaban Tervalidasi

Q1: Apa yang bisa `trace_id` lakukan yang tidak bisa dilakukan `request_id`?

`request_id` hanya memberi stempel pada baris log terpisah dalam satu service. `trace_id` merajut semua log dan span *melewati batasan service* menjadi satu diagram visual waterfall di Jaeger — satu ID yang sama tersebar di A, B, C, dan database.

Q2: Kenapa `POST /predictions/` selesai 172ms tapi total trace 5.5 detik?

Pola fire-and-forget: 172ms adalah waktu HTTP round-trip user (validasi → simpan pending → 202 Accepted). 5.5s adalah durasi `BackgroundTask` yang jalan paralel — user sudah dapat response, tapi Jaeger tetap mengawasi sampai background task selesai.

Q3: Kenapa `BatchSpanProcessor` lebih cocok untuk production?

`BatchSpanProcessor` mengumpulkan span di memori lokal, lalu mengirimnya ke Jaeger secara asinkron di background thread — tidak ada blocking pada request path. `SimpleSpanProcessor` mengirim langsung setiap span selesai, menyebabkan API berhenti sejenak menunggu network call ke Jaeger untuk setiap operasi.